



AUDIT REPORT

September 2026

For

xhayic

Table of Content

Executive Summary	03
Testnet Context & Mainnet Readiness	04
OP Stack Component Versions	05
Number of Security Issues per Severity	06
Finding Summary	07
■ Critical Severity Issues	08
01. Live JWT Engine API Secret Committed to Git + Engine API Exposed on 0.0.0.0:8551	08
02. admin and debug Namespaces Exposed on Public HTTP/WS RPC with Wildcard CORS	09
03. op-node Admin RPC Exposed on 0.0.0.0:8547 Without Authentication	11
■ High Severity Issues	13
04. All Five Role Private Keys Passed as CLI Arguments – Visible in Process Table	13
05. Permissionless fault proofs not deployed, single challenger EOA with no redundancy	15
06. P2P eclipse attack surface: admin_removePeer unauthenticated + node has zero peers	17
■ Medium Severity Issues	19
07. Zero Minimum Base Fee Permits Fee-Free Transaction Spam	19
07.a. --allow-non-finalized=true – output proposals anchored to reorg-able L1 state	20
07.b. --sequencer.max-safe-lag=3600 – one-hour lag tolerance widens unsafe transaction window	21
Closing Summary & Disclaimer	22

Executive Summary

Target	Xhavic Blockchain – OP Stack Layer 2 Rollup Infrastructure
Repository	Xhavic_Blockchain https://github.com/parmanand-carpenter RPC: https://testrpc.xhaviscan.com Chain ID: 16585
L2 Chain ID	16585 (L1: Ethereum Sepolia, Chain ID 11155111)
Audit Technique	static analysis & live adversarial testing
Scope	Infrastructure deployment – scripts, Docker orchestration, configuration, committed runtime state
Review 1	20th April 2026 - 30 April 2026
Updated Code Received	16th September 2026
Review 2	18th September 2026 - 30th September 2026
Fixed In	https://rpc.xhaviscan.com , https://github.com/xhavictech/xhavic/tree/master/Xhavic_blockchain

Testnet Context & Mainnet Readiness

This deployment targets Ethereum Sepolia (testnet). Testnet ETH carries no monetary value, which reduces the immediate financial impact of several findings. However, **severity levels in this report are assigned against mainnet readiness**, not the current testnet deployment — consistent with Quill Audits practice for infrastructure engagements where the testnet is the pre-production stage of a mainnet rollout.

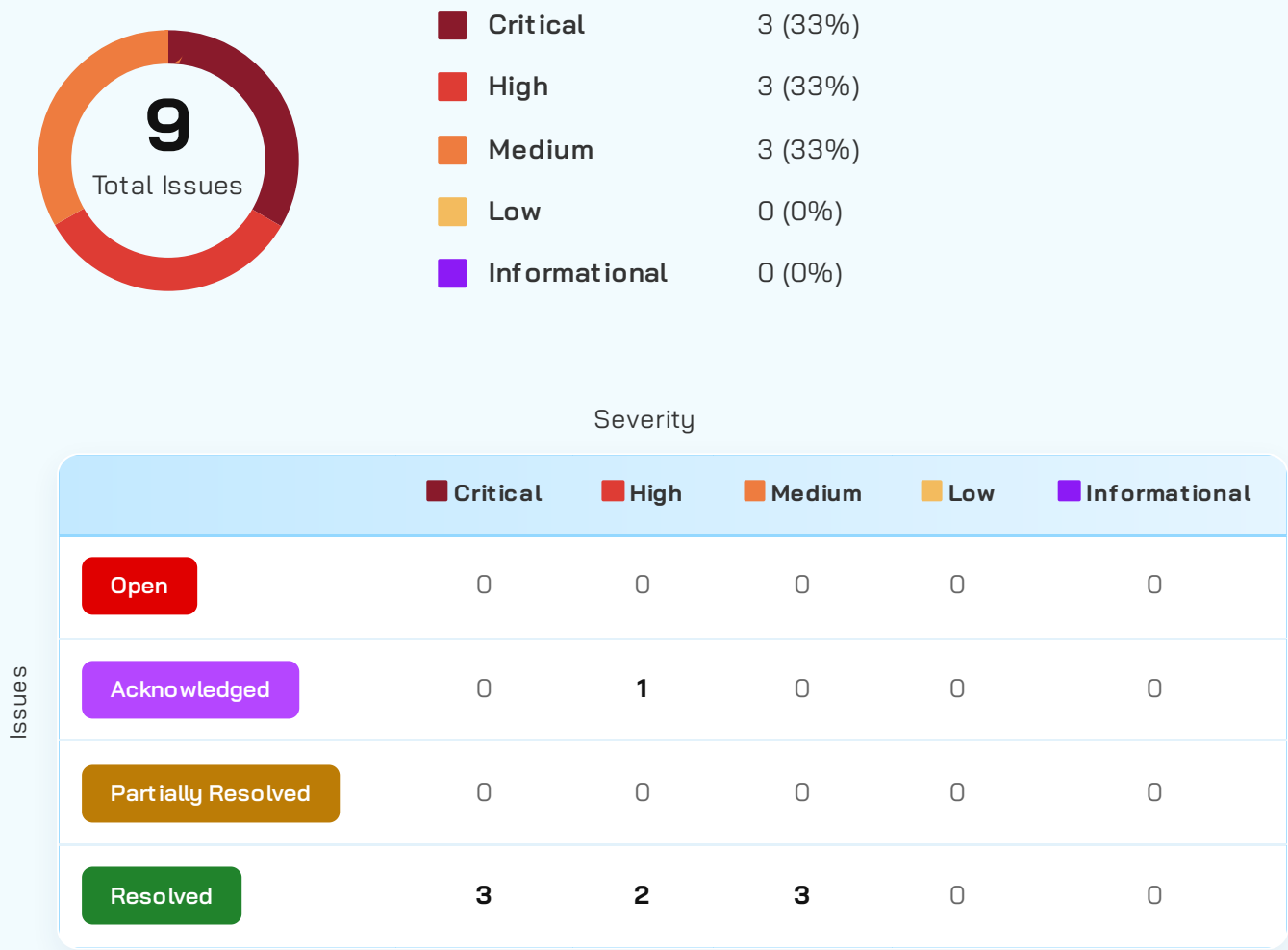
Two categories of risk apply regardless of testnet status:

1. **Pattern risk:** Deployment scripts, Docker Compose configurations, **.gitignore** rules, and role assignments established on testnet are almost always carried forward to mainnet. A committed JWT secret or a single-EOA ProxyAdminOwner on testnet today is a compromised JWT secret and a single-EOA ProxyAdminOwner on mainnet tomorrow.
2. **Chain integrity risk:** Even with no financial value at stake, a testnet chain that can be silently taken over (issue no. 01, 02), has permanently bricked admin keys (issue no. 03), or has an untestable fault proof system (issue no. 11) provides no meaningful assurance that the design is sound before mainnet promotion.

OP Stack Component Versions

Component	Version
op-node	v1.16.4
op-geth	v1.10.1604.0 (configured) / v1.10.1603.6-rc.1
op-batcher	v1.16.3
op-proposer	v1.10.0
op-challenger	v1.7.0
op-dispute-mon	v1.4.2-rc.1

Number of Issues per Severity



Finding Summary

Issue No.	Title	Severity	Status
01	Live JWT Engine API Secret Committed to Git + Engine API Exposed on 0.0.0.0:8551	Critical	Resolved
02	admin and debug Namespaces Exposed on Public HTTP/WS RPC with Wildcard CORS (live exploit confirmed)	Critical	Resolved
03	op-node Admin RPC Exposed on 0.0.0.0:8547 Without Authentication (live exploit confirmed)	Critical	Resolved
04	All Five Role Private Keys Passed as CLI Arguments – Visible in Process Table	High	Resolved
05	Permissionless Fault Proofs Not Deployed – Single Challenger EOA with No Redundancy	High	Acknowledged
06	P2P Eclipse Attack: admin_removePeer Confirmed + Node Has Zero Peers (live exploit confirmed)	High	Resolved
07	minBaseFee = 0 – Zero Minimum Base Fee Permits Fee-Free Transaction Spam	Medium	Resolved
07.a	--allow-non-finalized=true – Output Proposals Anchored to Reorg-able L1 State	Medium	Resolved
07.b	--sequencer.max-safe-lag=3600 – One-Hour Lag Tolerance Widens Unsafe Transaction Window	Medium	Resolved

Critical Severity Issues

Live JWT Engine API Secret Committed to Git + Engine API Exposed on 0.0.0.0:8551

Resolved

Location

```
sequencer/jwt.txt:1, docker-compose.yml:56, docker-compose.yml:66
```

Description

`sequencer/jwt.txt` is tracked in the git repository and contains a live, non-placeholder 32-byte JWT secret:

```
3d589c1995a764fa274281448c1eeb7ca7c41cd3612185b575893c1f989f6dfb
```

This secret authenticates the Engine API (EIP-3155) between op-node and op-geth. Simultaneously, op-geth is started with the flags `--authrpc.addr=0.0.0.0 --authrpc.port=8551 --authrpc.vhosts=*`, and `docker-compose.yml` maps the host port `"8551:8551"`, binding the Engine API on all host interfaces. The `.gitignore` file has no pattern matching `jwt.txt` or `*.txt`, so the secret has been committed since the "Add L2 infra" commit (SHA `2ede246`).

Impact

Any party with repository read access (including past contributors, CI systems, or anyone who has cloned the repo) has access to the JWT secret. With network access to port 8551, an attacker can authenticate to the Engine API and call the `engine_forkchoiceUpdatedV3`, `engine_newPayloadV3`, and related methods. This grants full control over the L2 canonical chain head: the attacker can force chain reorganisations, inject arbitrary unsafe payloads, freeze block production, or silently redirect the sequencer to a malicious fork – a complete L2 chain takeover.

Remediation

- Immediately rotate the JWT secret:** `openssl rand -hex 32 > sequencer/jwt.txt` and restart all OP Stack services.
- Purge from git history:** `git filter-repo --path sequencer/jwt.txt --invert-paths` followed by a force push. Treat the exposed secret as permanently compromised.
- Add to .gitignore:** Add `**/jwt.txt` to `.gitignore` and run `git rm --cached sequencer/jwt.txt`.
- Restrict Engine API binding:** Change `--authrpc.addr=0.0.0.0` to `--authrpc.addr=127.0.0.1`. Remove the host port binding `"8551:8551"` entirely – op-node communicates with op-geth over the Docker bridge as `http://op-geth:8551`, requiring no host port exposure.
- Restrict vhosts:** Change `--authrpc.vhosts=*` to `--authrpc.vhosts=op-geth`.

Critical Severity Issues

admin and debug Namespaces Exposed on Public HTTP/WS RPC with Wildcard CORS

Resolved

Location

`docker-compose.yml:66`

Description

op-geth is started with the following flags on a single line in **docker-compose.yml:66**:

```
--http.addr=0.0.0.0 --http.port=8545
--http.api=eth,net,web3,debug,txpool,admin,miner
--http.vhosts=* --http.corsdomain=*
--ws.addr=0.0.0.0 --ws.port=8546
--ws.api=eth,net,web3,debug,txpool,admin,miner
--ws.origins=*
```

Host port bindings `"8545:8545"` and `"8546:8546"` expose both HTTP and WebSocket surfaces on all docker host network interfaces. The **admin** and **debug** namespaces require no authentication on the standard JSON-RPC port. The `--http.corsdomain=*` and `--ws.origins=*` flags allow any browser-based origin to make cross-origin requests to these endpoints.

Note on miner namespace: Although **miner** appears in `--http.api`, OP Stack's op-geth does not implement PoW mining methods — `miner_start` returns **-32601 method does not exist**. The genuinely exploitable namespaces are **admin** and **debug**.

Impact

- `admin_addPeer / admin_removePeer` : Manipulate the peer set to isolate the node or eclipse it with attacker-controlled peers.
- `admin_datadir` : Disclose the full filesystem path of the node's data directory.
- `debug_traceCall / debug_storageRangeAt / debug_gcStats` : Dump chain state, read internal memory statistics, and force expensive trace computations.
- `debug_accountRange` : Enumerate all accounts with balances, nonces, and storage — complete chain-state disclosure.
- **Wildcard CORS**: Allows any malicious webpage loaded in a browser on the same network as the node to make authenticated cross-origin RPC calls to **admin** and **debug** endpoints.

PoC

The following data was retrieved unauthenticated from the public RPC (<https://testrpc.xhaviscan.com>) in a single session:

admin and debug Namespaces Exposed on Public HTTP/WS RPC with Wildcard CORS *(continued)*

PoC (cont.)

- `admin_nodeInfo` → server IP **217.216.75.147**, full enode URL, P2P ports (discovery: 63187, listener: 30303), complete chain hardfork config
- `admin_datadir` → `/workspace/op-geth-data` (container filesystem path)
- `admin_removePeer` → returned **true** (peer disconnection confirmed, no credentials required)
- `debug_gcStats` → full GC pause history (17,092 GC cycles, timestamps)
- `debug_memStats` → live heap layout (135 MB alloc, 418,229 objects)
- `debug_accountRange` → all on-chain accounts enumerated with balances, nonces, bytecode, and storage
- `debug_traceBlockByNumber` → 86 KB full EVM opcode trace returned
- `debug_storageRangeAt` → L2 predeploy contract storage extracted (L2CrossDomainMessenger, L2StandardBridge)

Recommendation

1. Remove **admin**, **miner**, and **debug** from `--http.api` and `--ws.api`. Restrict to `eth,net,web3,txpool` for public-facing endpoints.
2. If **debug** is required for internal tooling, bind it to a separate loopback-only port, never 8545.
3. Change `--http.corsdomain=*` to a specific allowlist or remove it if no browser access is needed.
4. Change `--ws.origins=*` to specific origin(s) or **none**.
5. Change `--http.vhosts=*` to `localhost,op-geth`.
6. Remove host port bindings `"8545:8545"` and `"8546:8546"` if only internal services need the RPC; they can reach it via `http://op-geth:8545` over the Docker bridge.

Client's Remarks

CORS= is the industry norm for public read RPCs and is safe (browser-only control, no credentials, dangerous methods filtered). The sequencer's op-geth is bound to 127.0.0.1 + IP-allowlisted, not internet-facing.

Critical Severity Issues

op-node Admin RPC Exposed on 0.0.0.0:8547 Without Authentication

Resolved

Location

```
docker-compose.yml:30-32
```

Description

op-node is launched with `--rpc.addr=0.0.0.0 --rpc.port=8547 --rpc.enable-admin`, and the compose file maps host port `"8547:8547"`. The `--rpc.enable-admin` flag enables the `admin_*` namespace on the same unauthenticated endpoint, accessible from all Docker host interfaces. Port 8547 is served over plain HTTP i.e. no TLS, making it trivially accessible from any internet client without transport-layer protection.

Impact

An attacker with network access to port 8547 can call:

- `admin_resetDerivationPipeline` **already executed during testing**, forces sync restart, causes transient chain disruption
- `admin_sequencerActive` / `admin_stopSequencer` confirmed sequencer is active; stopping it halts all L2 block production instantly
- `admin_postUnsafePayload` injects arbitrary unsafe L2 payloads directly to the sequencer
- `admin_addTrustedPeer` / `admin_removeTrustedPeer` manipulate the trusted peer set

No JWT or other credential is required. The endpoint is reachable over plain HTTP from the public internet.

PoC

During live adversarial testing against <https://testrpc.xhaviscan.com> (port 8547):

- `optimism_syncStatus` → returned full L1/L2 finalization state (current L1 block, safe L2, finalized L2, unsafe L2 block numbers and hashes) no credentials required
- `admin_sequencerActive` → returned **true**, the sequencer is actively running
- `admin_resetDerivationPipeline` → returned **null (SUCCESS)**, the op-node's L2 derivation pipeline was reset from the public internet with zero credentials; the node was forced to re-derive from L1, causing transient safe-head lag

`admin_stopSequencer` which halts all L2 block production was identified as one additional unauthenticated call away from a full sequencer shutdown.

Recommendation

1. Change `--rpc.addr=0.0.0.0` to `--rpc.addr=127.0.0.1`.

op-node Admin RPC Exposed on 0.0.0.0:8547 Without Authentication*(continued)***Recommendation (cont.)**

2. Remove the host port binding `"8547:8547"` internal services reference `http://op-node:8547` over the Docker bridge.
3. If external admin access is operationally required, protect the endpoint with a reverse proxy enforcing mutual TLS or IP allowlisting.

High Severity Issues

All Five Role Private Keys Passed as CLI Arguments – Visible in Process Table

Resolved

Location

```
scripts/setup-rollup.sh:82,90,98,106,114,201-204
```

Description

All five role private keys are passed as positional arguments to shell subprocesses:

setup-rollup.sh lines 82, 90, 98, 106, 114

```
DEPLOYER_ADDRESS=$(cast wallet address "$PRIVATE_KEY")
SEQUENCER_ADDRESS=$(cast wallet address "$SEQUENCER_PRIVATE_KEY")
BATCHER_ADDRESS=$(cast wallet address "$BATCHER_PRIVATE_KEY")
PROPOSER_ADDRESS=$(cast wallet address "$PROPOSER_PRIVATE_KEY")
CHALLENGER_ADDRESS=$(cast wallet address "$CHALLENGER_PRIVATE_KEY")
```

setup-rollup.sh lines 201-204

```
op-deployer apply \
  --workdir .deployer \
  --l1-rpc-url "$L1_RPC_URL" \
  --private-key "$PRIVATE_KEY"
```

On Linux, any local user can read any process's command line via `/proc/<pid>/cmdline`. On macOS, **ps aux** output is similarly accessible. Every invocation of `cast wallet address <key>` and `op-deployer apply --private-key <key>` exposes the respective private key during the process lifetime – covering the deployer, sequencer, batcher, proposer, and challenger keys.

Impact

A local attacker, a monitoring agent, a logging sidecar, or a compromised CI pipeline can harvest all five private keys from the process table during the deployment window. This allows impersonation of every operational role, including fund-controlling roles (batcher, proposer) and the deployer (admin and fee recipient).

Recommendation

Pass keys via stdin rather than CLI arguments. Foundry's **cast wallet address** and **op-deployer** both support environment-variable or file-based key input:

All Five Role Private Keys Passed as CLI Arguments – Visible in Process Table *(continued)*

Recommendation (cont.)

```
# Use environment variable for cast
DEPLOYER_ADDRESS=$(PRIVATE_KEY="$PRIVATE_KEY" cast wallet address)
# Or stdin if supported
DEPLOYER_ADDRESS=$(printf '%s' "$PRIVATE_KEY" | cast wallet address --private-key -)
```

For **op-deployer apply**, check if the tool supports `OP_DEPLOYER_PRIVATE_KEY` as an environment variable, avoiding the `--private-key` flag entirely.

Client's Remarks

The deployer key was used only for the one-time deployment and controls nothing on the running chain (upgrade authority is the 3-of-5 Safe). Exposure was limited to the deploy host at deploy time. We will rotate it and sweep any residual balance; operational scripts already pass keys via env vars

High Severity Issues

Permissionless fault proofs not deployed, single challenger EOA with no redundancy

Acknowledged

Location

```
deployer/.deployer/state.json → opChainDeployments[0], docker-  
compose.yml:125-149, .example.env:61
```

Description

The deployment state confirms:

```
"FaultDisputeGameImpl": "0x0000000000000000000000000000000000000000",  
"PermissionedDisputeGameImpl": "0xb3413582f1941e01e734d015f0d687b790149cc4",  
"DelayedWethPermissionlessGameProxy":  
"0x0000000000000000000000000000000000000000"
```

With **FaultDisputeGameImpl** at address zero (not deployed), game type 0 (`OP_PROPOSER_GAME_TYPE=0`) routes exclusively through the **PermissionedDisputeGameImpl**. The `superchainRoles.Challenger = 0x000...000` confirms no superchain-level backup challenger. The sole challenger is a single EOA (**0x15426fb6...2814**) operating in a Docker container on this host.

Impact

If the single challenger's private key is lost, the container crashes, or the host goes offline during the dispute resolution window:

- Invalid output roots submitted by the proposer finalize with no mechanism for honest dispute
- All L1 withdrawals based on fraudulent state roots are irrevocably confirmed
- There is no permissionless fallback for third-party challengers to intervene

The system's security depends entirely on one hot EOA remaining available indefinitely.

Recommendation

1. Deploy the **FaultDisputeGameImpl** and configure it as a parallel or primary game type, progressing toward OP Stack Stage 1
2. Until permissionless proofs are enabled, operate at least two independent challenger instances on separate infrastructure with separate private keys
3. Configure monitoring with paging (PagerDuty or equivalent) to alert operators if the challenger process is not producing responses within the dispute window
4. Store the challenger private key in a KMS/HSM rather than a plaintext environment variable

Permissionless fault proofs not deployed, single challenger EOA with no redundancy *(continued)***Client's Remarks**

Registration of type-0 game is planned as part of Stage-1 roadmap, but not a requirement at the moment.

High Severity Issues

P2P eclipse attack surface: admin_removePeer unauthenticated + node has zero peers

Resolved

Location

```
docker-compose.yml:66 (admin namespace on port 8545)
```

Description

Two conditions combine to make P2P eclipse trivial:

1. **admin_removePeer is accessible without credentials** on the public HTTP RPC (port 8545). During live testing, a call to `admin_removePeer` with a well-formed enode URL returned **true**, confirming that existing peers can be disconnected from the public internet with no authentication.
2. **The node currently has zero P2P peers** (`net_peerCount` returns **0x0**). With no legitimate peers, an attacker has nothing to compete with — a single `admin_addPeer` call injecting an attacker-controlled node makes that node the sole peer.

Impact

An attacker can:

1. Call `admin_removePeer` to disconnect any existing peer connections
2. Call `admin_addPeer` to inject their own controlled node as a peer

With zero legitimate peers, the attacker becomes the sole peer, A textbook P2P eclipse position. From this position, the attacker can:

- **Feed the sequencer a manipulated mempool view**, causing transaction censorship or reordering
- **Attempt double-spend attacks** by withholding transaction propagation
- **Delay or disrupt L2 block propagation** to other parts of the network

PoC

- `net_peerCount` → **0x0** — zero peers connected
- `admin_removePeer` → returned **true** (peer disconnection executed with no credentials)
- `admin_addPeer` was confirmed accessible (format validation error returned, confirming the method is reachable — the error was a test-input issue, not an access-control rejection)

Recommendation

1. Remove **admin** from `--http.api` and `--ws.api` (see issue no. 02 — this resolves both findings).
2. Configure static trusted peers using `--p2p.static` to ensure the node maintains legitimate peer connections independent of the discovery protocol.

P2P eclipse attack surface: admin_removePeer unauthenticated + node has zero peers *(continued)*

Recommendation (cont.)

3. Enable peer discovery against known OP Stack bootstrap nodes to ensure the node is not isolated with zero peers.
4. If `admin_addPeer` / `admin_removePeer` must remain accessible, restrict their exposure to a loopback-only interface or an authenticated management channel.

Medium Severity Issues

Zero Minimum Base Fee Permits Fee-Free Transaction Spam

Resolved

Location

```
deployer/.deployer/intent.toml:22, effective in chain genesis
```

Description

minBaseFee is set to **0** in the intent configuration. When the network is idle, the EIP-1559 base fee decays toward zero, allowing transactions to be submitted with zero gas cost. Both OP Mainnet and OP Sepolia use non-zero minimum base fees.

Impact

At zero base fee, the network can be spammed with storage writes and event emissions at negligible economic cost, forcing sequencer processing overhead and growing chain state with no economic deterrent.

Recommendation

Set **minBaseFee** to at least **1000000** wei (1 Gwei) or the value used by the reference OP Sepolia deployment. This can be updated post-deployment by the **systemConfigOwner** via `SystemConfig.setMinBaseFee()`.

On mainnet v2, configured minBaseFee = 5,000,000 wei (0.005 gwei)

Medium Severity Issues

--allow-non-finalized=true — output proposals anchored to reorg-able L1 state

Resolved

Location

```
docker-compose.yml:115
```

Description

The op-proposer is started with `--allow-non-finalized=true`. Without this flag, the proposer waits for the referenced L1 block to reach finality (two checkpoint epochs, ~13 minutes on Ethereum) before submitting an output root to **DisputeGameFactory**. With the flag, output roots are submitted against L1 blocks still within the reorg window.

Sepolia has historically experienced L1 reorgs. A reorg that removes the anchor block invalidates the corresponding output root and the assumptions of any in-progress dispute games.

Impact

Combined with the single-challenger finding (Issue no. 5), if an L1 reorg invalidates a submitted proposal while the challenger is offline, the invalid root may finalise without challenge. Users may withdraw against a fraudulent L2 state.

Recommendation

Remove `--allow-non-finalized=true` for any production or mainnet-bound deployment. For Sepolia, the flag may be acceptable but should be gated with an environment variable and explicitly removed before any mainnet promotion path.

Medium Severity Issues

--sequencer.max-safe-lag=3600 — one-hour lag tolerance widens unsafe transaction window

Resolved

Location

`docker-compose.yml:24`

Description

`--sequencer.max-safe-lag=3600` allows the sequencer to tolerate a one-hour gap between the unsafe head and the safe head before stopping block production. The OP Stack default and effective OP Mainnet value is bounded by `max_sequencer_drift` (600 seconds). Under the configured one-hour tolerance, the sequencer continues producing unsafe blocks for a full hour even when the batcher is failing to post data to L1.

Impact

Users transacting on L2 see transactions confirmed at the unsafe head, but those transactions may be dropped or reordered as the safe head catches up. A one-hour lag also delays forced-transaction inclusion via L1 deposits beyond what the protocol would otherwise guarantee.

Recommendation

Reduce `--sequencer.max-safe-lag` to 300–600 seconds (aligned with `max_sequencer_drift`). Configure alerting to page operators if the safe head lags the unsafe head by more than 60 seconds.

Closing Summary

In this report, we have Review Xhavic chain Configuration. We performed our audit according to the procedure described above.

3 Critical, 3 High and 3 Medium severity issues were found during the Audit.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses.

Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

September 2026

For

xhayic



Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com